

Name: Ethan Jia

Student ID: ethanjia (2235723)

1. Raw audio is a time-domain waveform, which shows how the sound amplitude changes over time. However, keyword spotting depends heavily on frequency patterns, such as which frequencies are present and how they change while a word is spoken. By converting raw audio into spectrogram or MFCC-style features, the model receives a more useful representation of the sound.

A spectrogram shows both time and frequency information, while MFCC features provide a compact representation of the important speech characteristics. This makes it easier for the keyword spotting model to distinguish words like “yes” and “no.” MFCC features also reduce the input size and remove less useful information, which is important for running the model on a small embedded device like the Arduino Nano 33 BLE.

2. During full integer quantization, the representative dataset is used to show TensorFlow Lite the typical range of input values that the model will see during inference. The converter runs sample inputs through the model to estimate the minimum and maximum activation values for different layers.

This is important because an INT8 model stores weights and activations using 8-bit integers instead of 32-bit floating-point numbers. To do this correctly, TensorFlow Lite needs proper scale and zero-point values. If the representative dataset is not similar to the real input data, the quantized model may lose accuracy because the integer ranges will not match the actual audio features. A good representative dataset helps the INT8 model stay accurate while using less memory and running more efficiently on the Arduino.

3. `audio_processor.get_data()` prepares and returns audio feature data for the model. It takes raw audio data, processes it into the feature representation expected by the keyword spotting model, and provides the processed input data for training, evaluation, or inference.

Based on `input_data.py`, this function handles the audio preprocessing pipeline, including selecting the correct audio sample, applying any needed preprocessing such as background noise mixing or time shifting, and converting the audio into model-ready features such as spectrogram or MFCC-style input. In other words, it is the function that turns raw audio examples into the formatted input tensor that the neural network can use.

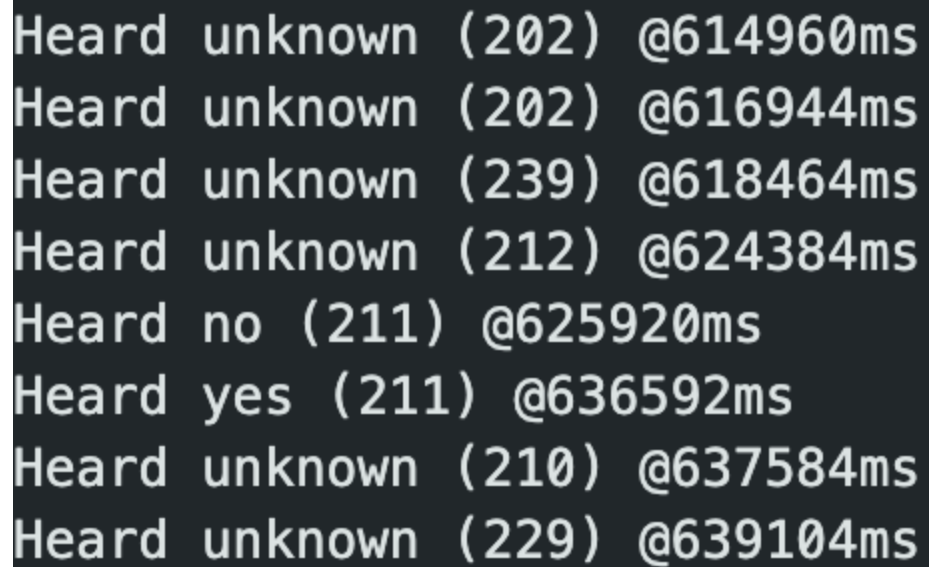
4. In the final export step, the TensorFlow Lite model is converted into a C/C++ byte array so it can be included directly in the Arduino source code. The Arduino does not load a `.tflite` file from a normal file system, so the model must be compiled into the program itself.

`g_model` is the byte array that stores the entire quantized TensorFlow Lite model in C/C++ format. It contains the model architecture, weights, and other data needed by TensorFlow Lite Micro.

`g_model_len` is the length of that byte array. It tells the Arduino program how many bytes are in the model.

Both must be copied correctly into `micro_features_model.cpp`. If the byte array is incomplete, corrupted, or if `g_model_len` does not match the actual model size, TensorFlow Lite Micro may fail to load the model or the keyword spotting result may be incorrect. In this lab, my generated model length was: `const int g_model_len = 18984;`

5.

A screenshot of a dark background with white text. The text consists of nine lines, each representing a keyword identification result. Each line follows the format: 'Heard [keyword] ([count]) @[timestamp]ms'. The keywords are 'unknown', 'unknown', 'unknown', 'unknown', 'no', 'yes', 'unknown', 'unknown', and 'unknown'. The counts are 202, 202, 239, 212, 211, 211, 210, 229, and 229 respectively. The timestamps are 614960, 616944, 618464, 624384, 625920, 636592, 637584, 639104, and 639104 respectively.

Heard unknown (202) @614960ms
Heard unknown (202) @616944ms
Heard unknown (239) @618464ms
Heard unknown (212) @624384ms
Heard no (211) @625920ms
Heard yes (211) @636592ms
Heard unknown (210) @637584ms
Heard unknown (229) @639104ms

Fig. 1 Screenshot of showing the correct identification of keywords